SYSTEM DOCUMENTATION

Hotzonex Cloud

MikroTik WiFi hotspot management — vouchers, routers, selling, reports, profit, alerts and multi-business platform administration.

Version 1.0 · 18 October 2026

Hotzonex WiFi · Gorom Head Office, Juba, South Sudan
app.hotzonex.net · info@hotzonex.net · +211 924 904 216

Proprietary and confidential
© 2026 Hotzonex WiFi

Contents

1 Introduction 3
1.1 Purpose of this document 3
1.2 What the system does 3
1.3 Key facts 3
2 System architecture 5
2.1 Components 5
2.2 Multi-business design (tenancy) 6
2.3 How work reaches a router 6
3 Users and roles 7
4 Getting started 8
4.1 Signing up a new business 8
4.2 First-day checklist 8
4.3 Language, theme and help 8
5 Routers and sites 9
5.1 Adding a router 9
5.2 Tunnel server 10
5.3 Router jobs 10
6 Plans and vouchers 11
6.1 Plans 11
6.2 Generating and printing vouchers 12
6.3 Voucher life cycle 12
7 Selling and payments 13
7.1 Sell screen 13
7.2 Pre-printed cards and voids 13
7.3 Mobile money (Buy page) 13
7.4 Agents and cash-up 14
8 Check voucher 15
9 Reports, expenses and profit 16
9.1 Dashboard 16
9.2 Reports 16
9.3 Expenses & profit 17
9.4 Business settings 17

10 Alerts and notifications 18
10.1 WhatsApp (no API key) 18
10.2 Telegram 19
10.3 Subscription reminders in three languages 19
11 Platform administration 20
11.1 Businesses 20
11.2 Billing for Hotzonex Cloud 20
11.3 Subscription plans 20
12 Security 21
13 Deployment and operations 22
13.1 Web app 22
13.2 Supabase 22
13.3 Configuration 22
14 Backup and recovery 23
15 Troubleshooting 24
16 Development and maintenance 25
16.1 Repository 25
16.2 Quality checks 25
16.3 Changing the database 25
16.4 Translations 25
A Appendix — data model 26
B Appendix — server functions 27
C Appendix — glossary 28

1 Introduction

Hotzonex Cloud is a cloud platform for running MikroTik WiFi hotspot businesses: it creates and sells voucher codes, connects to the routers, records every sale, and shows owners exactly what they earn and spend — in English, Arabic or Kiswahili, on any phone or computer.

1.1 Purpose of this document

This document describes the complete Hotzonex Cloud system: what it does, how each part works, how the different users operate it day to day, how it connects to MikroTik routers, how it is secured, and how it is deployed, backed up and maintained. It is written for three readers:

- **Business owners and staff** — chapters 3 to 10 explain every screen and task.
- **The platform operator** (Hotzonex Cloud, who serves many businesses) — chapters 11 and 13 to 16.
- **Technical maintainers** — chapters 2, 12 and 14 to 17 and the appendices.

1.2 What the system does

Vouchers & routers

Connect MikroTik routers (public IP, WireGuard tunnel or setup script), define plans with time, validity, speed and data limits (MB/GB), generate voucher codes, push them to the routers and print A4 card sheets or thermal receipts.

Selling

One-tap cash sales on the Sell screen (retry-safe), pre-printed cards, mobile-money purchases confirmed by a cashier, agents with commission and credit limits, and a daily cash-up.

Insight

Live dashboard, daily/weekly/monthly reports, expenses by category and a clear profit line (revenue − commission − expenses), plus a **Check voucher** tool that answers “was this code used?” instantly.

Platform

Many independent businesses on one system, each seeing only its own data; self sign-up, subscription plans and invoices, WhatsApp/Telegram alerts, billing reminders in three languages, support mode and business removal.

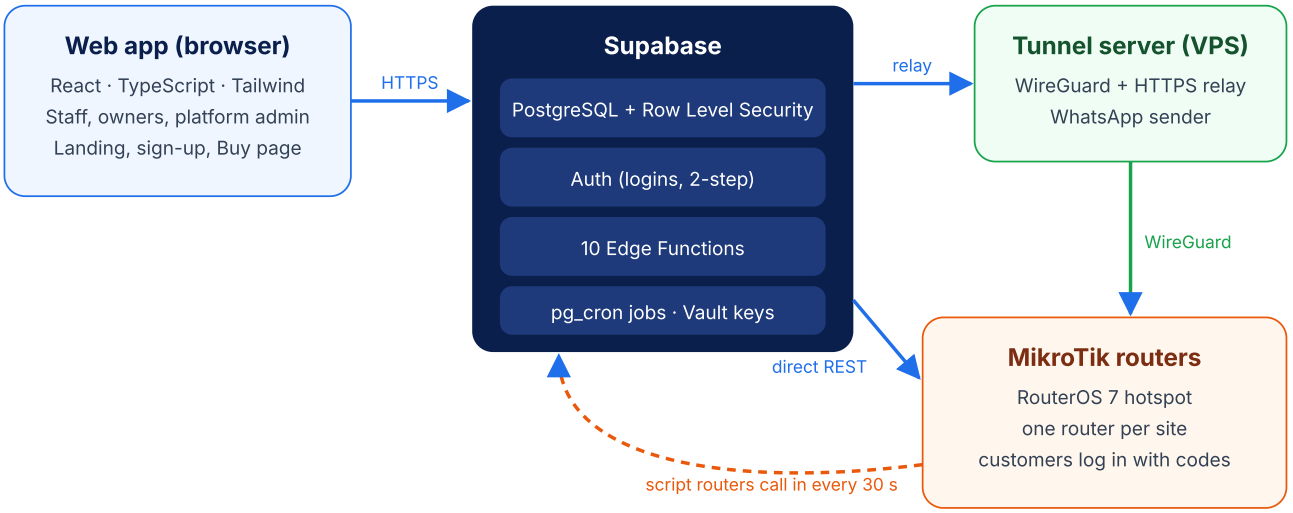
1.3 Key facts

Item	Value
Live address	**https://app.hotzonex.net**
Operator	Hotzonex WiFi — Gorom Head Office, Juba, South Sudan
Support	WhatsApp +211 924 904 216 · info@hotzonex.net

Item	Value
Currency and time	Whole South Sudanese Pounds (SSP), optional USD display; times in Africa/Juba (UTC+2)
Languages	English, العربية (Arabic, right-to-left), Kiswahili
Routers supported	MikroTik RouterOS 7 (7.1+ direct, 7.19+ script, any RouterOS 7 with WireGuard for the tunnel)
Web hosting	DigitalOcean App Platform (static site, automatic deploys)
Database, logins, server functions	Supabase (PostgreSQL 17, Auth, Edge Functions, Vault, pg_cron)

2 System architecture

Hotzonex Cloud has four parts: the web app in the browser, the Supabase back end (database, logins and server functions), the routers at each site, and an optional small tunnel server (VPS) for routers without a public IP and for WhatsApp alerts.



2.1 Components

Component	Technology	Role
Web app	React 18, TypeScript, Vite 7, Tailwind CSS 3, TanStack Query, Zod	All screens for staff, owners and the platform admin; public landing page, sign-up, Buy page and Terms. Installed as an app on phones (PWA) and opens fast from its own cache.
Database	Supabase PostgreSQL 17	All data. Every table is protected by Row Level Security; money and voucher changes go only through checked database functions.
Logins	Supabase Auth	Email and password, optional 2-step login (authenticator app), idle logins end after 8 hours.
Edge Functions	Deno (TypeScript) on Supabase	Work that needs a server: talking to routers, creating staff, deleting with router clean-up, alerts, public Buy page and sign-up.
Scheduler	pg_cron + pg_net	Wakes the router worker every minute, scans alerts every 2 minutes, runs billing and reminders daily.
Tunnel server	Ubuntu VPS, WireGuard, Caddy, Node 20	Optional. Reaches routers behind private addresses and sends WhatsApp alerts from a linked number.

Component	Technology	Role
Routers	MikroTik RouterOS 7	Run the hotspot. Hotzonex adds voucher users and reads usage; it never changes the hotspot set-up itself.

2.2 Multi-business design (tenancy)

Every business (tenant) has its own sites, routers, plans, vouchers, sales, staff, expenses and settings. Each row carries the business it belongs to, filled in automatically by the database, and a restrictive security policy guarantees that a login only ever sees and changes rows of its own business. The platform admin can enter a customer's business only through **support mode** (chapter 11), which is time-limited and fully logged.

2.3 How work reaches a router

When staff do something that affects a router (generate vouchers, save a plan, void a sale, install the login portal), the database adds a **router job** to a queue. The job is then delivered in one of three ways, depending on how the router was connected:

Connection	When to use	How work reaches it
Direct	Router has a public IP or DNS name	The `router-direct` function logs in to the router's REST API (password stored encrypted), at once when a job is queued and every minute.
Tunnel (recommended)	Router behind a private address (Starlink, 4G, CGNAT)	A one-time script adds a WireGuard link to the Hotzonex tunnel server; the router is then reached like a direct router through the relay.
Cloud script	No VPS; RouterOS 7.19+	A small scheduler on the router (`hzc-cloud`) calls `router-gateway` every 30 seconds and runs the commands it receives. Only a hashed token is stored.

Every minute each online router also reports who is online and how much time and data each voucher has used. The database uses this to show Active Users, record first logins, mark vouchers expired and count sales of activated cards.

3 Users and roles

Every login belongs to one business and has one role. The role decides the menu and, more importantly, what the database allows; hiding a menu item is never the only protection.

Role	Menu	Can do
Owner (admin)	Dashboard, Reports, Expenses & profit, Alerts, Sell, Payments, Sales, Check voucher, Cash-up, Vouchers, Plans, Routers, Active users, Login portal, Business settings, Staff, Billing, Security	Everything in the business, all sites; voids within 24 hours; disconnect users; whole-business expenses; commissions and credit limits.
Manager	Dashboard, Reports, Expenses & profit, Alerts, Sell, Payments, Sales, Check voucher, Cash-up, Vouchers, Active users, Security	Own site only: generate and print vouchers, sell, mark printed cards sold, reports and that site's expenses.
Cashier / agent (seller)	Sell, Payments, Sales, Check voucher, Cash-up, Security	Own site only: sell, print and share codes, confirm mobile-money payments, check vouchers, see own sales and cash. Agents earn a commission and may have a cash limit.
Platform admin	All of the above for their own business, plus **Businesses**	Runs Hotzonex Cloud itself: adds and supports businesses, billing, subscription plans, WhatsApp sender, billing messages, switch off and remove businesses. Requires 2-step login.

On phones a bottom tab bar shows the most used pages: Sell, Payments, Sales and Cash-up for cashiers; Dashboard, Sell, Payments and Vouchers for owners and managers. The menu (☰) has the rest, plus the help buttons (WhatsApp, Email), language and Light/Dark/Auto theme.

4 Getting started

4.1 Signing up a new business

1. Open **app.hotzonex.net** and go to the sign-up page (**Start your free trial**). Enter the business name, first site (optional), your name, phone number, email (your login) and a password of at least 8 characters. Starting the trial accepts the terms of service and privacy.
2. A short **welcome wizard** follows: business details and logo, first site, first router, first plan and first cards.
3. The trial (by default 14 days) starts at once. Reminders are sent before it ends (chapter 10).

4.2 First-day checklist

1. Log in as the owner. Turn on **2-step login** on the Security page (recommended).
2. **Routers → Sites**: add your sites (for example Gorom, Jebel Iraq, Sub-Office).
3. **Routers → Add router** for each site and follow the connection steps (chapter 5). The router must show **Online** with its model, RouterOS version and hotspot.
4. **Plans**: set prices, time, validity, speed, devices, data limit and colours (chapter 6).
5. **Staff**: create managers and cashiers, each with a site.
6. **Vouchers → Generate vouchers**; wait for the batch to show **Ready**.
7. **Print cards** for pre-printed stock and/or sell from the **Sell** screen.
8. Optional: **Payments → Buy page** for mobile money, **Alerts → WhatsApp** numbers, **Login portal** design.

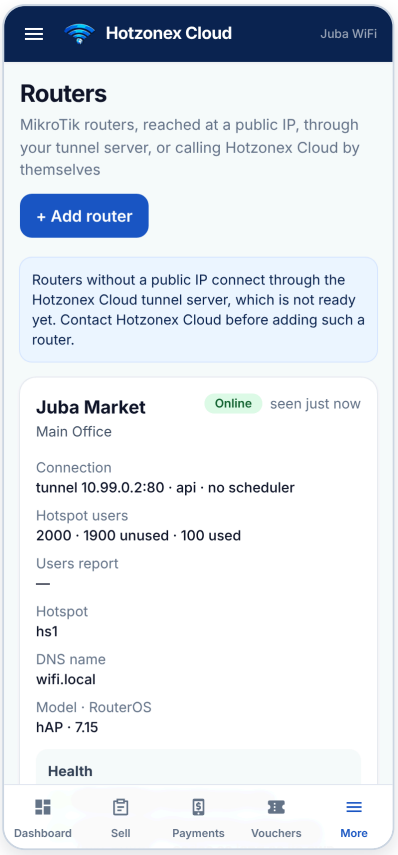
4.3 Language, theme and help

At the bottom of the menu each person chooses **English, العربية or Kiswahili** and **Light, Dark or Auto**; the choice is saved on that phone or computer. Arabic turns the whole app right-to-left. The **Help from Hotzonex Cloud** buttons open a WhatsApp chat or an email to support. Printed cards, receipts and reports are always printed light.

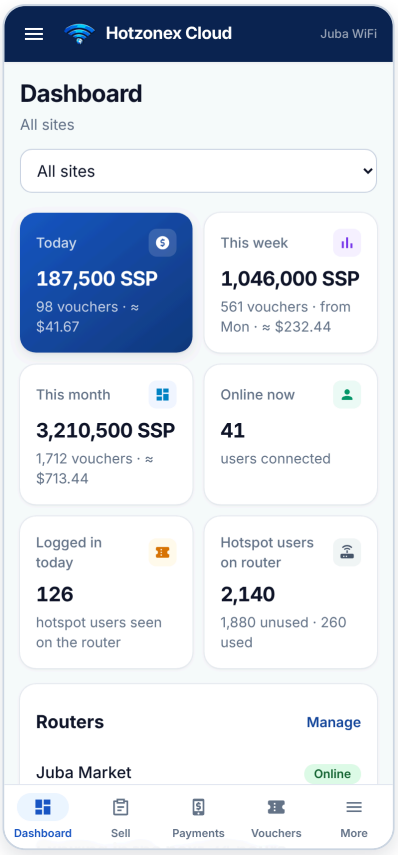
5 Routers and sites

First set up the hotspot on the router yourself (IP → Hotspot → Hotspot Setup). Hotzonex uses that hotspot exactly as it is and **never changes** the hotspot, its profiles, the firewall or other settings. What it writes is limited to:

- hotspot users for its vouchers, each tagged `comment=hzc:<voucher id>` — only tagged users are ever disabled or removed;
- its own user profile per plan (`hzc-…`), only where the router has no profile with that name;
- closing sessions (validity ended, voided sale, Disconnect), and the login portal files when you install them.



Routers page: connection, users and health



Dashboard: router status and stock

5.1 Adding a router

Routers → Add router, like a MikroTik app: Address, Port, Username, Password, Comment → **Connect**.

- **Public IP or DNS name**: Hotzonex logs in to the REST API directly. The `www` (80) or `www-ssl` (443, trusted certificate) service must be reachable from the internet. The password is stored encrypted (AES-256-GCM, key in Supabase Vault).
- **Private address** (for example 192.168.88.1): Hotzonex shows a one-time script to paste in Winbox → New Terminal. The script first checks that a hotspot is running (otherwise it changes nothing), then connects the router through the tunnel server, or installs the `hzc-cloud` scheduler when no tunnel server is set up.

The first check only **reads** the hotspot servers, server profiles and user profiles, picks the running hotspot server and its DNS name, and shows them under **Hotspot setup**.

5.2 Tunnel server

One small Ubuntu VPS (about US$4–6 a month) serves all routers. Allow UDP 51820 and TCP 80/443, then paste the install command from **Routers → Tunnel server**. It installs WireGuard and an HTTPS relay with a free certificate, keeps the router list in sync and registers itself (the card turns **Ready**). Each router then gets a WireGuard interface `wg-hzc`, an address `10.99.0.x` and one firewall rule, all tagged `hotzonex-tunnel`.

```
/ip firewall filter remove [find comment="hotzonex-tunnel"]
/ip address remove [find comment="hotzonex-tunnel"]
/interface wireguard remove [find name="wg-hzc"]
```

The three commands above remove the tunnel from a router if ever needed.

5.3 Router jobs

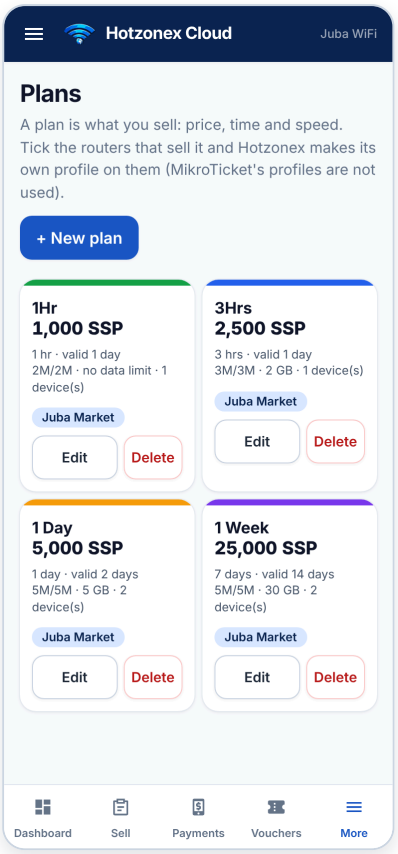
Job	When	What happens on the router
`test_connection`	Router added or edited, **Test again**	Reads hotspot servers and profiles; warns about a wrong server name, DNS name or missing plan profile. Changes nothing.
`sync_profiles`	Plan saved, new router passes its test	Adds the plan's own user profile (speed, devices) if missing; removes a deleted plan's profile when no user is on it.
`push_vouchers`	Vouchers generated, **Retry push**	Adds each code as a hotspot user: name = password = code, profile, server, `limit-uptime`, `limit-bytes-total`, tag comment.
`disable_voucher`	Sale voided, voucher deleted	Disables or removes the tagged user and closes its sessions.
`disconnect_user`	**Disconnect** on Active users	Closes that session.
`install_portal` / `restore_portal`	Login portal install / switch back	Downloads the portal pages into a `hotzonex` folder and points the hotspot to it, or back to the previous folder.

Every router write is safe to repeat. A job whose result does not come back is sent again (up to 3 times); if a router goes offline during a push, the push resumes when it is back. A router is shown offline after two missed checks, so a short internet drop does not flash it offline.

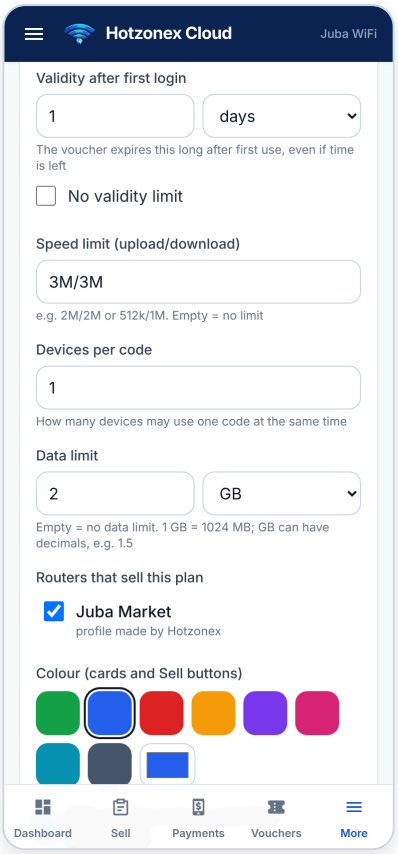
6 Plans and vouchers

6.1 Plans

A plan is what you sell. Each plan has a name, price (SSP), **time** (minutes, hours or days of internet), **validity after first login** (or none), **speed limit** (for example `2M/2M`), **devices per code**, **data limit** and a colour used on cards and Sell buttons. Tick the routers that sell the plan; Hotzonex creates its own profile there named `hzc-<plan>-<price>`.



Plans list



Plan form with the MB/GB data limit

Data limit in MB or GB

Choose MB or GB next to the data limit; GB accepts up to two decimals (1.5 GB = 1,536 MB). The limit is saved in MB and applied to each voucher on the router as `limit-bytes-total` = MB × 1,024 × 1,024 (1 GB → 1,073,741,824 bytes). Upload and download count together; when it is used up the router disconnects the customer and the voucher is marked expired. RouterOS user profiles have no data setting, which is why the limit lives on each voucher. A changed limit applies to vouchers generated afterwards.

> **Old MikroTicket plans**
> Plans that were made from MikroTicket profiles before 16 October 2026 are switched off and listed at the bottom of the Plans page so past reports keep their names. MikroTicket cards on the routers keep working and their sales are still counted.

6.2 Generating and printing vouchers

- **Vouchers → Generate vouchers**: choose router, plan, quantity (up to 1,000 per batch) and code length (6–10 digits). The batch is pushed to the router and shows **Ready**; a failed push resumes by itself or with **Retry push**.
- **Print cards**: A4 sheets with 24, 40 or 50 cards per page. Print at A4 portrait, scale 100 %, no headers/footers, with **background graphics** on. Printing marks the cards as printed (not sold); **Unmark these cards** undoes a failed print.
- **Receipts**: 58 mm or 80 mm thermal paper, remembered per device.
- **Vouchers list**: search by code, filter by router, plan, status and batch; delete unsold vouchers (removed from the router too).

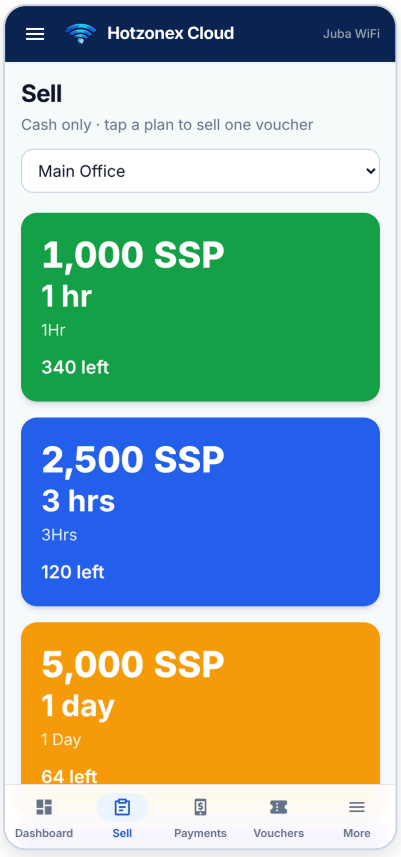
6.3 Voucher life cycle

Status	Meaning
Unused	On the router, never logged in, not sold.
Sold	Sold (Sell screen, mobile money or marked sold) but not logged in yet.
Active	Logged in at least once. First login sets “valid until” = first login + plan validity.
Expired	Time, validity or data used up. Removed from the router once offline.
Disabled	Switched off (voided sale or deleted voucher).

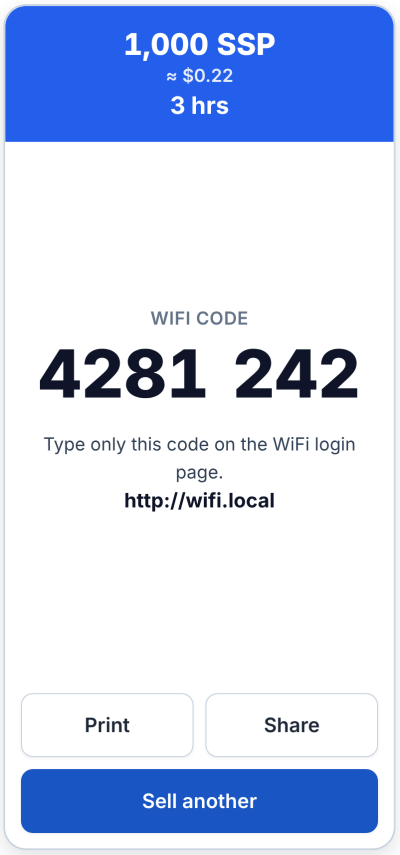
7 Selling and payments

7.1 Sell screen

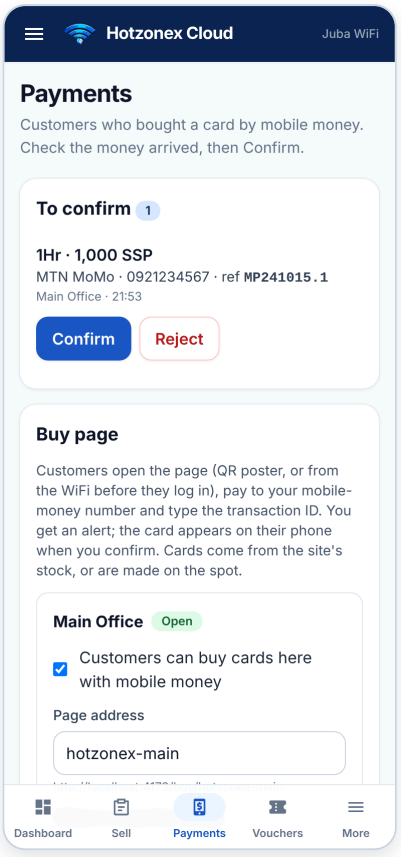
One big button per plan showing price, duration and cards left (yellow below 20, disabled at 0). Each tap sells one voucher and shows the code full-screen with **Print** and **Share** (WhatsApp or the phone's share sheet). Two cashiers can never get the same code, and if the connection drops during a sale the app retries with the same request and gets back the same card — a cashier can never sell two cards by accident.



Sell screen



The sold code with Print and Share



Mobile-money payments to confirm

7.2 Pre-printed cards and voids

- Pre-printed cards are recorded with **Sales → Mark printed cards as sold**, or automatically with **Sell on first login** for the site: the first login of an unused card records a cash sale.
- Every activated card counts as a sale, including MikroTicket cards (at the price of their plan); they appear with seller "Router (MikroTicket voucher)".
- **Voids**: owners only, within 24 hours, with a reason. The sale stays listed as voided, is left out of all totals, and the voucher is disabled on the router.

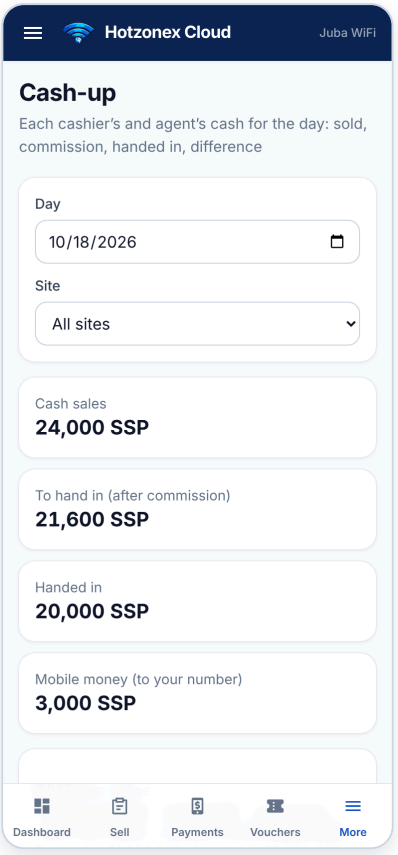
7.3 Mobile money (Buy page)

Payments → Buy page: switch it on for a site, choose its address and payment instructions (for example *MTN MoMo 092 … (Hotzonex WiFi)*) and print the QR poster. Customers can open the page before

logging in. They choose a card, send the money and type the transaction ID; the cashier gets an alert, checks the money arrived and taps **Confirm**, and the code appears on the customer's phone. A transaction ID works once; requests are limited per phone and device and expire after a day.

7.4 Agents and cash-up

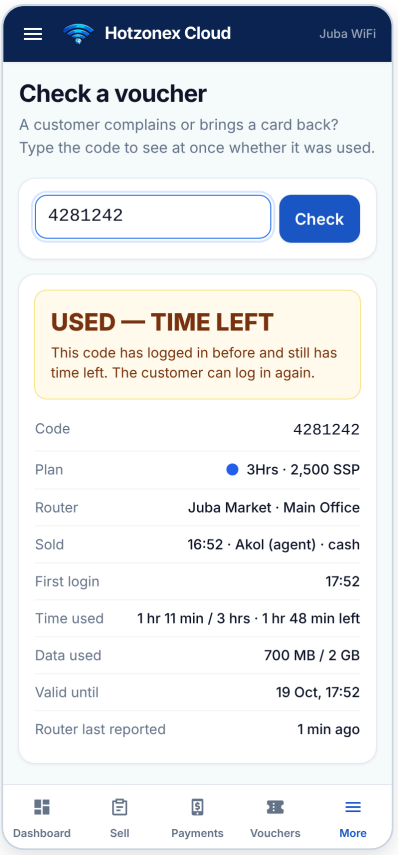
Cash-up shows per seller and day: cash sales, commission, cash to hand in, cash handed in, the difference and the cash each seller holds now. **Receive cash** records a hand-over. Owners set each agent's **commission** (0–50 %) and **credit limit**: above the limit the agent cannot sell until cash is handed in.



Cash-up

8 Check voucher

When a customer complains or brings a card back, any staff member opens **Check voucher**, types the code and immediately sees whether it was used. It works for Hotzonex vouchers and for MikroTicket codes on the routers; cashiers and managers see their own site only.



A used voucher with time left

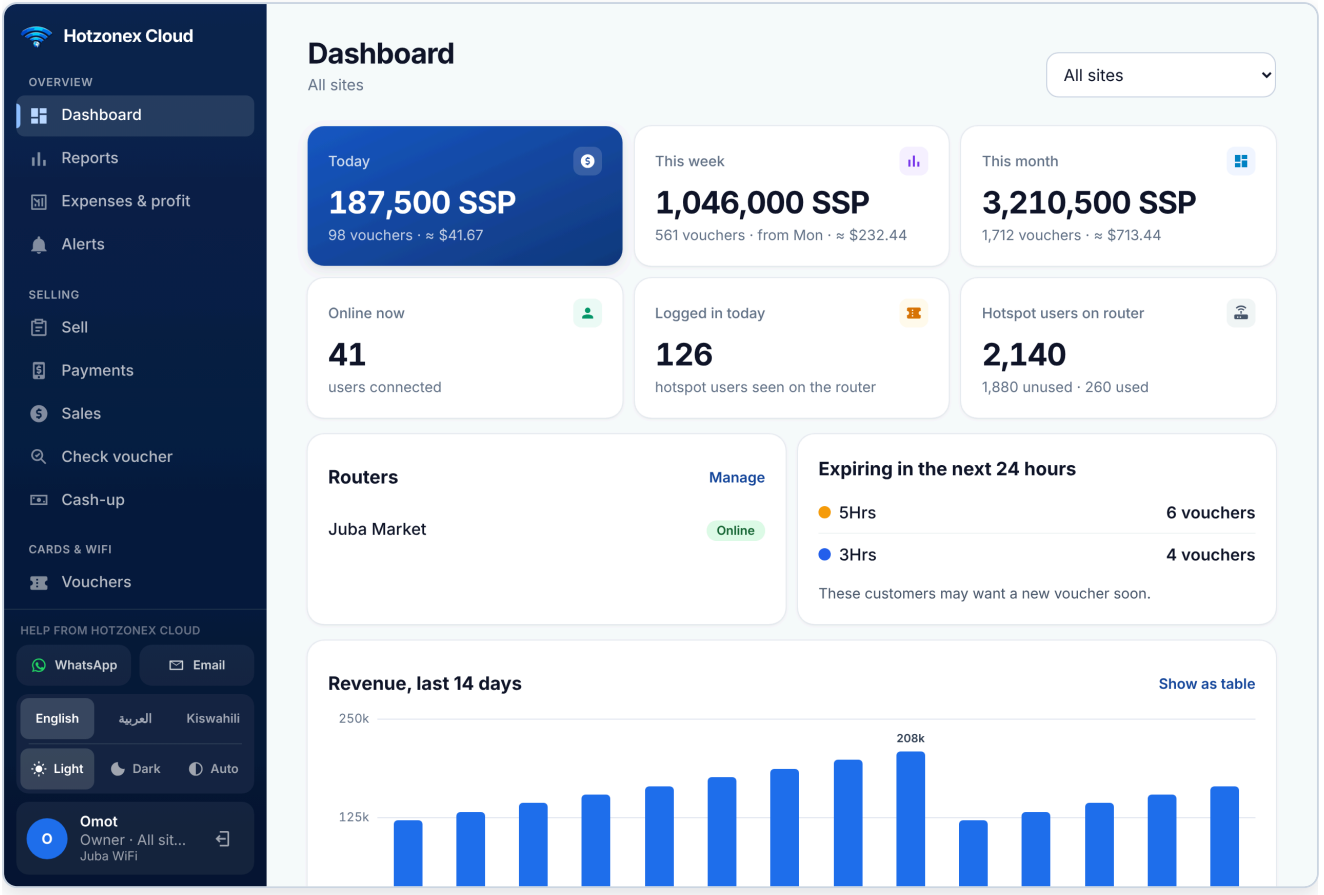
Answer	Meaning
NOT USED	The code has never logged in; it can still be used.
IN USE NOW	Someone is online with this code right now (device IP and MAC shown).
USED — TIME LEFT	It has logged in before and still has time; the customer can log in again.
USED UP	All time or data used, or validity ended; it cannot log in again.
SWITCHED OFF	Disabled in Hotzonex (for example a voided sale).

The details show plan and price, router and site, who sold it, when and how (cash or mobile money), first login, time used against the plan, data used against the limit, valid-until and when the router last reported. If the router has not reported for 10 minutes or is offline, a warning says the answer may be out of date.

9 Reports, expenses and profit

9.1 Dashboard

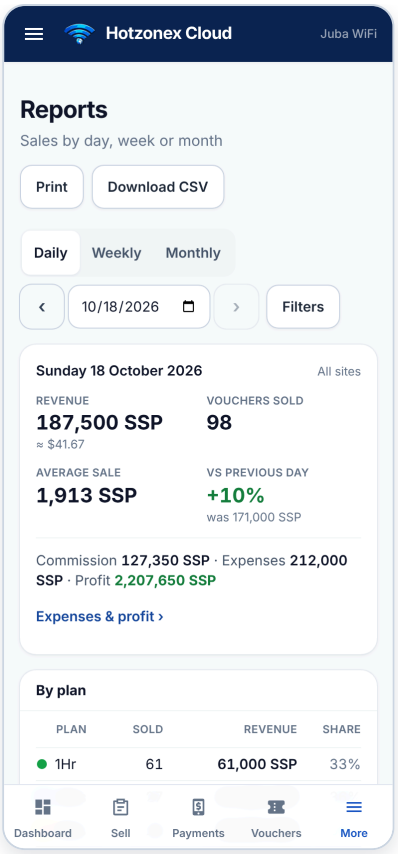
Today, this week (Monday–Sunday) and this month revenue, users online now, hotspot users on the routers (total, unused, used), logins today, revenue for the last 14 days, cards expiring in the next 24 hours, stock left per plan and site, and routers online/offline with last seen.



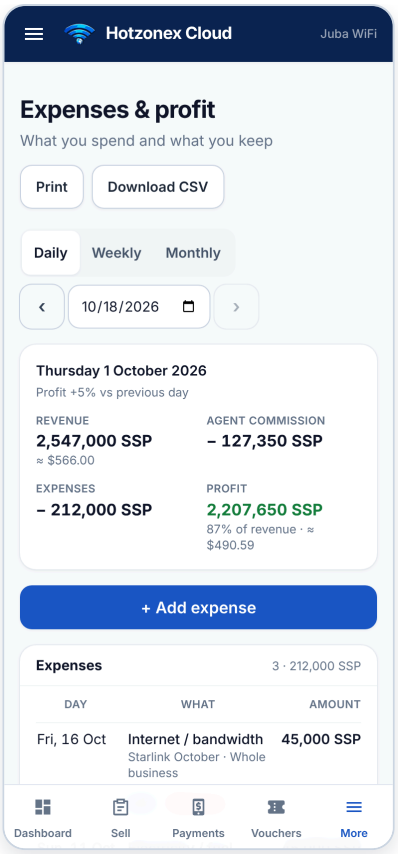
Dashboard on a computer

9.2 Reports

Daily, Weekly and Monthly tabs with filters for site, plan and seller: a summary with the change from the previous period, a **profit line** (revenue − agent commission − expenses), and breakdowns by plan, seller, site, hour or day, and voucher status. **Print** gives a clean A4 report; **Download CSV** opens in Excel. Voided sales are always excluded.



Reports



Expenses & profit

9.3 Expenses & profit

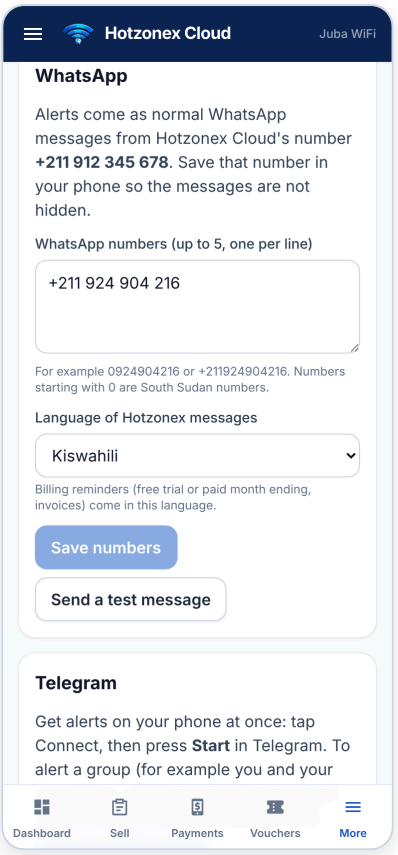
Owners and managers record running costs in the categories internet, power, rent, salaries, feeding, equipment, repairs, printing, transport and other — for one site or (owners) the whole business. The page shows revenue − commission − expenses = **profit** per day, week or month, day by day and per category, with Print and CSV. Every change is written to the audit log.

9.4 Business settings

Owners set the business name, logo (on cards, receipts and the Buy page) and the **dollar rate**; once set, USD is shown next to SSP on the dashboard, Sell screen and reports.

10 Alerts and notifications

Every 2 minutes Hotzonex looks for: a router offline for 5 minutes (and back online), cards that failed or need checking after a push, low stock, mobile-money payments to confirm, and billing messages. Alerts appear on the **Alerts** page and are sent by WhatsApp and/or Telegram.



Alerts → WhatsApp numbers and language

10.1 WhatsApp (no API key)

1. Put a **second SIM** (not a personal number) in a phone and install WhatsApp.
2. **Businesses → WhatsApp sender → Set up WhatsApp sender**: copy the command and paste it on the tunnel server (1–3 minutes, safe to repeat).
3. Scan the QR code from the sender phone: WhatsApp → Settings → Linked devices → Link a device. The card shows **Connected**.
4. Each business: **Alerts → WhatsApp** → up to 5 numbers (numbers starting with 0 are South Sudan) → **Save numbers** → **Send a test message**.

> **Good to know**
> This way of sending is unofficial (WhatsApp Web protocol). A few messages a day to people who expect them keeps the risk low, but WhatsApp can block a number it thinks is a bot — hence the separate SIM. Keep the sender phone charged and online at least every two weeks. Sending is limited (20 an hour per business; 3 tests per 10 minutes; 10 number changes a day) and failed messages are retried 5 times.

10.2 Telegram

Create a bot with **@BotFather**, add its token as the `TELEGRAM_BOT_TOKEN` secret in Supabase, then **Alerts → Connect Telegram** and press **Start** in Telegram (add the bot to a group first for group alerts).

10.3 Subscription reminders in three languages

Businesses are told before their service runs out: 7, 3 and 1 day before the paid month or free trial ends, plus new invoice, invoice due in 2 days, app paused (unpaid) and payment received. Each business picks **English, Arabic or Kiswahili** on Alerts → WhatsApp. Default texts are included; the platform admin can change them on **Businesses → Billing messages** with a live preview. Words such as `{business}`, `{date}`, `{days}`, `{amount}`, `{invoice}`, `{due}` and `{pay}` are filled in automatically.

11 Platform administration

The **Businesses** page is visible only to platform admins logged in with their 2-step code.

11.1 Businesses

- **Add business**: creates the business, first site and owner login.
- **Manage (support mode)**: work inside a customer's business as its admin for up to 2 hours to set up or fix routers. The customer is alerted and every action is logged under the platform admin's name.
- **Switch off / Switch on**: pauses a business — its staff cannot log in; nothing is deleted.
- **Remove** (only after switching off): type the business name to confirm. Its sites, routers, plans, vouchers, sales, expenses, invoices, alerts, staff logins and logo are deleted for good; the routers keep working on their own, disconnected from Hotzonex Cloud. A short record of each removal is kept.

11.2 Billing for Hotzonex Cloud

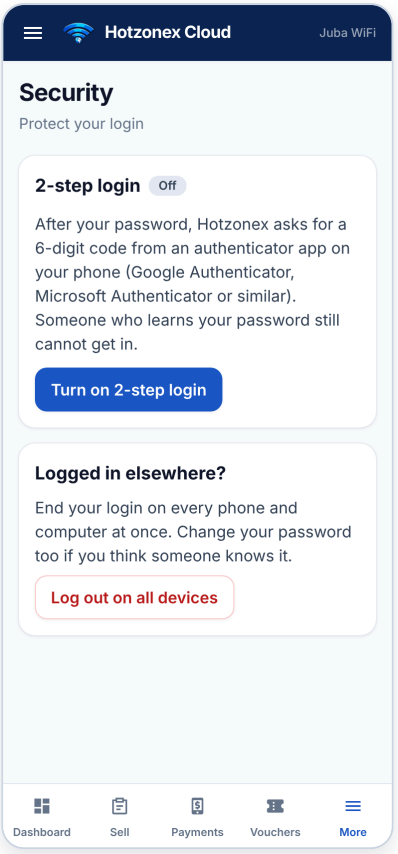
Set the **price per router per month**, grace days, trial days and payment instructions. Each morning an invoice is made per billed business (connected routers × price); reminders go out before the due day; unpaid after the grace days, the business is paused (its WiFi keeps working; admins see only the invoice and can submit the payment's transaction ID). **Mark paid** switches it back on. Businesses marked **Not billed** (such as Hotzonex's own) have no invoices and no limits.

11.3 Subscription plans

Plan	Routers	Sites	Staff	Vouchers a month
Starter	1	1	3	5,000
Business	5	3	15	30,000
Pro	no limit	no limit	no limit	no limit

Limits and prices are editable on **Businesses → Subscription plans**; a business can be moved to another plan or have its trial extended (+14 or +30 days). The database refuses to go over a plan's limits.

12 Security



Security page: 2-step login

- **Row Level Security** on every table; policies read the caller's business, role and site. Each business sees only its own rows.
- **Checked functions**: sales, voids, voucher status, cash hand-overs and settings change only through `SECURITY DEFINER` database functions that check the caller; the browser cannot insert sales or edit vouchers directly.
- **2-step login** (authenticator app; QR code or "Open in authenticator app"): once turned on, the database gives the login no access until the code is typed, so a stolen password alone opens nothing. Platform powers always require it.
- **Router secrets**: direct-router passwords and tunnel keys are encrypted with AES-256-GCM; the key is in Supabase Vault and readable only by the server. Script routers store only the SHA-256 of a random 256-bit token.
- **Audit trail**: who, what, when and which business for sales, voids, batches, plan and price changes, staff changes, routers, expenses, settings, billing plans and support-mode actions.
- **Web protection**: functions answer only the app's own websites (CORS); the site carries a Content-Security-Policy; idle logins end after 8 hours; **Log out on all devices** ends every login.
- **No secrets in the app**: the service-role key never leaves Supabase. Only the public (anon) key is in the web app, by design.

13 Deployment and operations

13.1 Web app

The web app is a static site on **DigitalOcean App Platform** (source directory `web`, build `npm run build`, output `dist`, catch-all document `index.html`). Every push to the deployment branch redeploys it automatically. The open app checks `/version.json` on start, every 5 minutes and when shown again, and loads a new version on the next page change; the service worker fetches the app from the network first and uses its saved copy only offline.

13.2 Supabase

- **Migrations** in `supabase/migrations` are applied in file-name order; an applied file is never edited — each change is a new file.
- **Edge Functions**: `add-router`, `create-staff` and `delete-items` are called by logged-in staff; `router-gateway`, `router-direct`, `tunnel`, `portal`, `alerts`, `buy` and `signup` check their own callers.
- **Authentication**: email provider on, public sign-up through the app's own sign-up function; Site URL and redirect URLs set to app.hotzonex.net. For real volumes add your own SMTP for emails.
- **Scheduled jobs** (pg_cron): router worker every minute, alert scan every 2 minutes, billing run and reminders daily.

13.3 Configuration

Where	Setting	Purpose
DigitalOcean (build)	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`	Optional: the app has Hotzonex Cloud's project built in.
Supabase secrets	`TELEGRAM_BOT_TOKEN`	Optional: turns on Telegram alerts.
Supabase Vault	`router_secret_key`	Created by a migration; encrypts router passwords and tunnel keys.
Web app code	`web/src/lib/contact.ts`	Public phone, WhatsApp, email and office shown in the app.

14 Backup and recovery

- **Database**: Supabase Pro projects have daily backups. On any plan keep your own copy at least weekly and before big changes (`supabase db dump` for schema and data).
- **Automatic daily copy** to DigitalOcean Spaces with `scripts/backup/hotzonex-backup.sh` run nightly from a Droplet; 30 days are kept. Restore by applying the migrations, then the `auth` dump, then the `public` dump.
- **The router password key** (`router_secret_key` in Vault) is not in the backup files. Read it once in the SQL Editor and store it offline; without it, router passwords must be re-entered and tunnel scripts run again on a new project.
- **Routers**: `/export file=hotzonex-backup` on each router. Vouchers on a reset router can be pushed again with **Retry push**.

15 Troubleshooting

What you see	What to check
“… is a private address …”	Direct needs a public IP or DNS name; otherwise use the tunnel or setup script.
“No reply from …” / “refused the connection”	Public IP correct, `www` / `www-ssl` on, port allowed in the router firewall, not behind CGNAT.
“Wrong router username or password”	The router login’s group needs read, write, api and rest-api.
“HTTPS certificate is not trusted”	Give the router a real certificate or switch HTTPS off.
Router stays “Waiting for router”	Script not pasted or no internet: `/system script run hzc-cloud`, `/log print where topics~"script"`; after an upgrade: `/certificate settings set builtin-trust-anchors=trusted`.
Router shows Offline	Script routers: no call for 2 minutes. Direct/tunnel: two missed checks; the reason is shown.
Codes do not log in	`/log print where topics~"hotspot"`. MikroTicket cards in Code mode: use “I have a username and password”.
Batch stuck on “Push failed”	Router offline; it resumes when back, or press **Retry push**.
Page shows “A new version is ready”	Refresh once; the app then loads the new version automatically.
WhatsApp alerts not arriving	Sender card shows Connected? On the VPS: `systemctl status hzc-wa`, `journalctl -u hzc-wa -n 50`. Keep the sender phone online.
Check voucher shows “may be out of date”	The router has not reported for 10 minutes: check that it is online.

16 Development and maintenance

16.1 Repository

Folder	Contents
`web/`	The React app: `src/features/*` pages, `src/components`, `src/lib` (data access, validation, formatting), `src/i18n` (English, Arabic, Kiswahili texts), service worker and build configuration.
`supabase/migrations`	76 SQL migrations: tables, security policies, database functions, schedules.
`supabase/functions`	The 10 Edge Functions and shared code (router protocols, tunnel, WhatsApp, billing messages) with tests.
`scripts/backup`	Nightly backup script for a Droplet.

16.2 Quality checks

```
npm run install:all   # web dependencies
npm run typecheck     # strict TypeScript (also checks every text exists in all 3 languages)
npm run lint          # ESLint, no warnings allowed
npm test              # router, billing, buy, data-limit and form logic
npm run build:web     # production build
```

All checks must pass before a push, because every commit on the deployment branch is built and published. Continuous integration runs the same checks on GitHub.

16.3 Changing the database

1. Write a new migration file (never edit an applied one).
2. Test it on a local database, then apply it to the live project (`supabase db push` or the SQL Editor).
3. Regenerate the web app's database types (`supabase gen types typescript …`).
4. Deploy any changed Edge Functions.

16.4 Translations

All screen texts live in `web/src/i18n/locales/*.ts`, one file per area. Every English text must have an Arabic and a Kiswahili version (the type check refuses a missing one). Server messages are translated by matching their English text. A native speaker should review the Arabic and Kiswahili wording from time to time.

A Appendix — data model

Table	Holds
`tenants`	Businesses: name, time zone, currency, logo, dollar rate, on/off.
`sites`	Sites of a business, Buy page settings, sell on first login.
`profiles`	Staff: name, role, site, active, commission, credit limit.
`routers`	Routers: connection type, address, encrypted login, status, hotspot information, user counts.
`plans` , `plan_routers`	Plans and which routers sell them (with the profile used).
`voucher_batches` , `vouchers`	Generated batches and every code with status, usage, first login and expiry.
`sales`	Every sale: voucher, seller, amount, method, commission, void details.
`router_logins`	Every hotspot user seen on the routers (MikroTicket and others), used or not.
`active_sessions`	Who is online now on each router.
`router_jobs`	Queue of work for the routers and its results.
`payment_requests`	Mobile-money purchases waiting for or after confirmation.
`cash_handovers`	Cash handed in by sellers.
`expenses`	Running costs by site and category.
`alerts` , `alert_settings`	Alerts and each business's channels, numbers and language.
`portal_configs`	Login portal design per router.
`tenant_billing` , `invoices` , `subscription_plans`	Billing state, invoices and plans with limits.
`platform_admins` , `removed_businesses`	Platform admins (and support mode) and a record of removed businesses.
`audit_log`	The audit trail.
`app_config`	Platform settings (billing, tunnel, WhatsApp sender, message templates).

B Appendix — server functions

Function	Called by	Purpose
`add-router`	Owners	Add and edit routers, setup and tunnel scripts, tunnel server set-up.
`create-staff`	Owners, platform admin	Create staff logins and new businesses.
`delete-items`	Owners, platform admin	Delete vouchers and plans with router clean-up; remove a business.
`router-direct`	Database (every minute)	Run jobs and read usage on direct and tunnel routers.
`router-gateway`	Script routers	Hand out jobs and receive results and usage.
`tunnel`	Tunnel server	Installer, WireGuard peer list, registration.
`portal`	Routers	Login portal pages.
`alerts`	Database, staff, Telegram	Send alerts (WhatsApp, Telegram), WhatsApp sender set-up, billing message templates.
`buy`	Public Buy page	Show plans, take mobile-money requests, hand the code over after confirmation.
`signup`	Public sign-up page	Create a new business with its owner and trial.

C Appendix — glossary

Term	Meaning
Business (tenant)	One hotspot company using Hotzonex Cloud, fully separated from the others.
Site	A location of a business, usually with one router.
Plan	What is sold: price, time, validity, speed, devices, data limit.
Voucher / code	The digits a customer types to log in to the WiFi.
Validity	How long a voucher stays usable after its first login, even if time is left.
Profile	The router's hotspot user profile holding speed and devices per code.
Push	Sending generated vouchers to the router as hotspot users.
Tunnel	The WireGuard link that lets Hotzonex reach a router without a public IP.
Support mode	A platform admin temporarily acting inside a customer business, logged.
RLS	Row Level Security: database rules deciding which rows each login may see or change.
MikroTicket	Another voucher tool; its cards keep working and are counted by Hotzonex.

Hotzonex Cloud is proprietary software. Copyright © 2026 Hotzonex WiFi. All rights reserved. Screens in this document use sample data.